



Explain SOQL in detail with multiple examples and explain all the examples

Absolutely—let's dive **deep into SOQL (Salesforce Object Query Language)**.

I'll explain:

- ✓ What SOQL is
- ✓ Syntax & concepts
- ✓ Multiple examples, each with clear explanations

What is SOQL?

SOQL (Salesforce Object Query Language) is:

- A query language like SQL, but specifically for Salesforce data.
- Used to retrieve data stored in Salesforce objects (e.g., **Account**, **Contact**).
- Read-only: you can **SELECT**, but not **INSERT/UPDATE/DELETE** with SOQL.
- Often used in Apex, API integrations, and Developer Console.

Basic SOQL Syntax

pgsql

```
SELECT fields
FROM ObjectName
[WHERE condition]
[ORDER BY field [ASC|DESC]]
[LIMIT n]
```

Examples with Explanations

★ 1. Retrieve All Accounts

Query:

soql

```
SELECT Id, Name FROM Account
```

✓ **Explanation:**

- **SELECT Id, Name** retrieves the Id and Name fields.
- **FROM Account** queries all records in the **Account** object.

- No WHERE clause = fetch **all Account records**.

2. Filter Records with WHERE Clause

Query:

```
soql

SELECT Id, Name FROM Contact WHERE LastName = 'Smith'
```

Explanation:

- Retrieves Contacts whose **LastName** is exactly "Smith".
- WHERE clause filters your results.
- Good for finding specific customer records.

3. Use Comparison Operators

Query:

```
soql

SELECT Id, Name, AnnualRevenue FROM Account WHERE AnnualRevenue > 500000
```

Explanation:

- Returns Accounts where Annual Revenue exceeds 500,000.
- Comparison operator > filters higher revenue customers.

4. ORDER BY Clause

Query:

```
soql

SELECT Id, Name, CreatedDate FROM Lead ORDER BY CreatedDate DESC
```

Explanation:

- Fetches Leads sorted by creation date in **descending order** (newest first).
- Helps with showing recent leads.

5. LIMIT Clause

Query:

```
soql
```

```
SELECT Id, Name FROM Account ORDER BY CreatedDate DESC LIMIT 5
```

✓ Explanation:

- Fetches the **5 most recently created Accounts**.
- Useful for dashboards or previews.

★ 6. LIKE Operator (Wildcard Search)

Query:

```
soql
```

```
SELECT Id, Name FROM Contact WHERE Email LIKE '%.edu'
```

✓ Explanation:

- Returns Contacts with email addresses ending in ".edu".
- % is a wildcard matching any characters.

★ 7. IN Operator

Query:

```
soql
```

```
SELECT Id, Name FROM Account WHERE Industry IN ('Technology', 'Healthcare')
```

✓ Explanation:

- Returns Accounts in either Technology



Yes

Great—let's **continue** with more **SOQL examples** and detailed explanations. We stopped at Example 7 (IN Operator). I'll pick up right there:

★ 7. IN Operator (continued)

Query:

```
soql
```

```
SELECT Id, Name FROM Account WHERE Industry IN ('Technology', 'Healthcare')
```

✓ Explanation:

- Returns all Accounts where **Industry** is either *Technology* or *Healthcare*.
- **IN** operator checks membership in a set of values.
- Equivalent to:

```
java
```

```
WHERE Industry = 'Technology' OR Industry = 'Healthcare'
```

★ 8. NOT IN Operator

Query:

```
soql
```

```
SELECT Id, Name FROM Contact WHERE MailingState NOT IN ('CA', 'TX')
```

✓ Explanation:

- Returns Contacts whose mailing state **is not California or Texas**.

★ 9. NULL Checks

Query:

```
soql
```

```
SELECT Id, Name FROM Opportunity WHERE CloseDate = NULL
```

⚠ **Important:** In SOQL, you must use `= NULL` or `!= NULL` to check for nulls.

✓ Explanation:

- Returns Opportunities where **CloseDate is blank** (not set).

★ 10. Aggregate Functions

Query:

```
soql
```

```
SELECT COUNT() FROM Contact WHERE CreatedDate = LAST_N_DAYS:30
```

✓ Explanation:

- `COUNT()` counts the total number of Contacts **created in the last 30 days**.
- Use for reporting totals.

★ 11. GROUP BY

Query:

```
soql

SELECT LeadSource, COUNT(Id)
FROM Lead
GROUP BY LeadSource
```

✓ Explanation:

- Groups Leads by their **LeadSource** (e.g., Web, Phone).
- Returns each LeadSource and how many leads came from it.
- Useful for summary reports.

★ 12. WITH ROLLUP

Query:

```
soql

SELECT Industry, COUNT(Id)
FROM Account
GROUP BY ROLLUP(Industry)
```

✓ Explanation:

- Adds a grand total row showing the **total count across all industries**.

★ 13. Relationship Queries – Child to Parent (Parent Fields)

When you want parent fields while querying a child object.

Query:

```
soql

SELECT Id, FirstName, LastName, Account.Name
FROM Contact
```

✓ Explanation:

- Returns Contacts with their associated Account's Name.

- `Account.Name` is the parent field.

★ 14. Relationship Queries – Parent to Child (Subquery)

When you want child records in one query.

Query:

```
soql

SELECT Name, (SELECT LastName FROM Contacts)
FROM Account
WHERE Name = 'Acme'
```

✓ **Explanation:**

- Fetches:
 - The Account with Name = 'Acme'
 - All Contacts related to that Account (subquery in parentheses).
- Parent-to-child relationship.

★ 15. Date Literals

Salesforce has convenient date literals:

- `TODAY`
- `YESTERDAY`
- `LAST_WEEK`
- `NEXT_N_DAYS:30` (next 30 days)

Query:

```
soql

SELECT Id, Name FROM Opportunity WHERE CloseDate >= LAST_N_DAYS:90
```

✓ **Explanation:**

- Opportunities closing in the **last 90 days**.

★ 16. ORDER BY Multiple Fields

Query:

```
soql

SELECT Name, AnnualRevenue, CreatedDate
FROM Account
```

ORDER BY AnnualRevenue DESC, CreatedDate ASC

✓ Explanation:

- Orders Accounts by:
 - Highest revenue first
 - For same revenue, oldest CreatedDate first

★ 17. LIMIT with OFFSET

Query:

```
soql

SELECT Id, Name FROM Contact ORDER BY CreatedDate DESC LIMIT 10 OFFSET 10
```

✓ Explanation:

- Skips the first 10 records.
- Returns the next 10.
- Useful for **pagination**.

★ 18. USING SCOPE

Query:

```
soql

SELECT Name FROM Lead USING SCOPE MyTerritory
```

✓ Explanation:

- Limits query to records in a **specific Salesforce territory** or view.
- Often used in list views.

★ 19. Polymorphic Fields (WhoId / WhatId)

Query:

```
soql

SELECT Id, Subject, WhoId, WhatId
FROM Task
WHERE ActivityDate = TODAY
```

✓ Explanation:

- Tasks often relate to **WhoId** (Contact or Lead) and **WhatId** (Account, Opportunity, etc.).
- This query shows today's tasks with related records.

★ 20. SOSL vs. SOQL

Quick note:

- **SOQL** = Structured queries (SELECT ... FROM ... WHERE ...)
- **SOSL** = Full-text search across multiple objects.

Example SOSL (not SOQL):

```
sosl

FIND 'Smith' IN NAME FIELDS RETURNING Contact(Id, FirstName, LastName)
```

🔑 SOQL Tips

- ✓ Always select only the fields you need (performance).
- ✓ Use **indexed fields in WHERE** for faster queries.
- ✓ Avoid querying too many records without LIMIT (Governor Limits).
- ✓ Use relationship queries to avoid multiple queries in Apex.

✓ If you'd like, I can give you:

- More examples for complex joins.
- Performance optimization strategies.
- Examples of using SOQL inside Apex code.

Just let me know!



Explain SOSL in detail with multiple examples and explain all the examples

Absolutely—let's **deep dive into SOSL (Salesforce Object Search Language)**.

I'll cover:

- ✓ What SOSL is
- ✓ Syntax & concepts
- ✓ Multiple examples, each with clear explanations

🎯 What is SOSL?

SOSL is:

- **Salesforce Object Search Language**
- Used for performing **text searches across multiple objects and fields** at once
- Good for:
 - Searching across all records quickly
 - Full-text searches (e.g., any field that contains “John”)
- Different from SOQL:
 - **SOQL** queries 1 object at a time (structured query)
 - **SOSL** can search **multiple objects and fields simultaneously**
- Returns **lists of lists** of sObjects (1 list per object queried)

Basic SOSL Syntax

```
pgsql

FIND 'searchString'
[IN ALL FIELDS | NAME FIELDS | EMAIL FIELDS | PHONE FIELDS]
[RETURNING objectList]
[WITH DivisionFilter]
[LIMIT n]
```

Keywords:

- **FIND** specifies the text to search for.
- **IN** defines which fields to search.
- **RETURNING** defines which objects and fields to return.
- **LIMIT** caps the number of records returned per object.

★ Examples with Explanations

★ 1. Basic Text Search Across All Objects

Query:

```
sosl

FIND 'Acme'
```

✓ Explanation:

- Searches **all searchable fields** in all searchable objects for the word “Acme”.
- Returns records wherever “Acme” appears (e.g., Account Name, Contact Company, etc.).
- You rarely use this unscoped in production because it’s too broad.

☀ 2. Search Specific Fields: NAME FIELDS

Query:

```
sosl

FIND 'Smith' IN NAME FIELDS
```

✓ Explanation:

- Searches **only Name fields** (e.g., Contact.Name, Account.Name).
- More efficient than searching all fields.

☀ 3. Returning Specific Objects and Fields

Query:

```
sosl

FIND 'United'
RETURNING Account(Id, Name), Contact(Id, FirstName, LastName)
```

✓ Explanation:

- Searches **all fields** for “United”.
- Returns:
 - Matching Accounts with Id and Name
 - Matching Contacts with Id, FirstName, LastName
- Each object is returned as a separate list of sObjects.

☀ 4. Using Wildcards

Query:

```
sosl

FIND 'Uni*'
RETURNING Account(Name), Contact(FirstName, LastName)
```

✓ Explanation:

- * matches any characters after “Uni” (e.g., United, Universal).
- Wildcards can only be at the **end** of the search term.

☀ 5. Search in Specific Field Types

Query:

```
sosl

FIND 'Salesforce'
IN EMAIL FIELDS
RETURNING Contact(Email, FirstName, LastName)
```

✓ Explanation:

- Searches **only Email fields** across objects.
- Returns Contacts with Email containing “Salesforce”.

★ 6. Search with LIMIT

Query:

```
sosl

FIND 'John'
RETURNING Contact(Id, FirstName, LastName LIMIT 5), Lead(Id, Name LIMIT 3)
```

✓ Explanation:

- Searches for “John”.
- Limits:
 - Contacts: maximum 5 results
 - Leads: maximum 3 results

★ 7. Search with Division Filter

If your org has divisions:

Query:

```
sosl

FIND 'Acme'
WITH Division = 'North America'
RETURNING Account(Name)
```

✓ Explanation:

- Searches only records in the **North America division**.

★ 8. Search in Apex Code

SOSL is often used inside Apex.

Example Apex Code:

```
apex

List<List<SObject>> searchLists = [FIND 'Acme' RETURNING Account(Id, Name), Contact(Id, Name)];

List<Account> accounts = (List<Account>) searchLists[0];
List<Contact> contacts = (List<Contact>) searchLists[1];
```

✓ Explanation:

- `searchLists` is a list of lists.
 - `searchLists[0]`: Accounts
 - `searchLists[1]`: Contacts
- You cast each sub-list to the appropriate type.

★ 9. SOSL with Multiple Words (logical OR)

Query:

```
sosl

FIND 'Acme OR Global'
RETURNING Account(Name)
```

✓ Explanation:

- Returns Accounts containing **either** “Acme” **or** “Global” in any searchable field.

★ 10. Search Across Multiple Objects Without RETURNING Clause

⚠ **Note:** It returns records of all searchable objects but fewer details.

Query:

```
sosl

FIND 'Smith'
```

✓ Explanation:

- Searches all fields in all objects.
- Returns basic info.
- Generally not recommended—better to use RETURNING clause.

★ 11. SOSL Example with Phone Fields

Query:

```
sosl
```

```
FIND '415'
IN PHONE FIELDS
RETURNING Contact(Phone, FirstName, LastName)
```

✓ Explanation:

- Searches Contacts with phone numbers containing “415” (e.g., area code).

🌟 12. Search with Exact Phrase

Query:

```
sosl
```

```
FIND '\"Acme Corporation\"'
RETURNING Account(Name)
```

✓ Explanation:

- \"Acme Corporation\" searches for the **exact phrase**, not separate words.

🔑 Key Differences from SOQL

Feature	SOQL	SOSL
Search Scope	One object at a time	Multiple objects at once
Search Type	Structured field queries	Full-text search
Return Type	Single list of sObjects	List of lists (one per object)
Wildcards	LIKE with %	* at end of term
Aggregation	Supports COUNT(), GROUP BY	No aggregation functions

🧠 When to Use SOSL?

- ✓ You don’t know which object/field contains the value.
- ✓ You need to search multiple objects at once.
- ✓ You need partial text matches (full-text search).
- ✓ You want better performance searching text in large datasets.

✓ Want more examples or help writing Apex code to process SOSL results?

Just tell me—I’ll be glad to help further!



Explain DML in detail with multiple examples and explain all the examples

Absolutely—let's **deep dive into DML (Data Manipulation Language) in Salesforce Apex**.

I'll cover:

- ✓ What DML is
- ✓ All DML operations
- ✓ Syntax and Apex usage
- ✓ Multiple examples, each clearly explained

What is DML?

DML stands for **Data Manipulation Language**.

👉 In Salesforce Apex, DML statements let you:

- **Insert** records
- **Update** records
- **Upsert** (insert or update) records
- **Delete** records
- **Undelete** records (restore from Recycle Bin)
- **Merge** records (combine duplicates)

You can perform DML via:

- **DML Statements**

```
apex
insert myAccount;
```

- **Database Class Methods**

```
apex
Database.insert(myAccount, allOrNone);
```

Note: Both approaches perform similar operations, but **Database** methods provide finer control over partial success and error handling.

All DML Operations & Examples

I'll give you **multiple examples** for each operation.

🌟 1. INSERT

- ✓ **Purpose:** Create new records.

Example 1: Insert a Single Record

```
apex
```

```
Account acc = new Account(Name='Acme Inc');
insert acc;
```

Explanation:

- Creates a new Account with the Name “Acme Inc.”
- After `insert`, `acc.Id` is automatically populated.

Example 2: Insert Multiple Records

```
apex
```

```
List<Contact> contacts = new List<Contact>{
    new Contact(FirstName='John', LastName='Doe'),
    new Contact(FirstName='Jane', LastName='Smith')
};
insert contacts;
```

Explanation:

- Creates 2 Contact records in one call (bulk insert).

★ 2. UPDATE

✓ **Purpose:** Modify existing records.

Example: Update an Existing Record

```
apex
```

```
Account acc = [SELECT Id, Name FROM Account WHERE Name='Acme Inc' LIMIT 1];
acc.Name = 'Acme Corporation';
update acc;
```

Explanation:

- Retrieves an Account named “Acme Inc.”
- Changes its name to “Acme Corporation.”
- `update` writes the change to the database.

★ 3. UPSERT

✓ **Purpose:** Insert if record does not exist, or update if it does.

🔑 You must specify:

- An external ID field
- Or rely on `Id`

Example: Upsert Based on External Id

```
apex

Account acc = new Account(
    Name = 'Acme USA',
    External_Id__c = 'EXT123'
);
upsert acc External_Id__c;
```

Explanation:

- If an Account exists with **External_Id__c** = 'EXT123', it gets updated.
- Otherwise, a new Account is inserted.

Example: Upsert a List

```
apex

List<Account> accounts = new List<Account>{
    new Account(Name='Global Corp', External_Id__c='EXT200'),
    new Account(Name='Global Corp 2', External_Id__c='EXT201')
};
upsert accounts External_Id__c;
```

Explanation:

- Upserts two Accounts using **External_Id__c**.

★ 4. DELETE

✓ **Purpose:** Move records to Recycle Bin.

Example: Delete a Single Record

```
apex

Account acc = [SELECT Id FROM Account WHERE Name='Acme Corporation' LIMIT 1];
delete acc;
```

Explanation:

- Finds the Account and deletes it.
- Record goes into Recycle Bin.

Example: Delete Multiple Records

```
apex

List<Contact> contacts = [SELECT Id FROM Contact WHERE LastName='Doe'];
delete contacts;
```

Explanation:

- Deletes all Contacts with LastName "Doe."

5. UNDELETE

✓ **Purpose:** Restore records from Recycle Bin.

Example:

```
apex

List<DeletedRecord> recs = [SELECT Id FROM Account WHERE IsDeleted = true ALL ROWS];
undelete recs;
```

Explanation:

- Retrieves soft-deleted Accounts using **ALL ROWS**.
- Restores them to active status.

6. MERGE

✓ **Purpose:** Combine up to 3 records into 1 master record (for Leads, Accounts, Contacts).

Example: Merge Duplicate Accounts

```
apex

Account master = [SELECT Id FROM Account WHERE Name='Global Corp' LIMIT 1];
Account duplicate = [SELECT Id FROM Account WHERE Name='Global Corporation' LIMIT 1];
merge master duplicate;
```

Explanation:

- Merges **duplicate** into **master**.
- Child records (Contacts, Opportunities) move to the master.
- Duplicate record is deleted.

DML Exception Handling

If a DML operation fails, it throws a **DmlException**.

Example with Try-Catch:

```
apex

try {
    Account acc = new Account(); // Missing Name (required)
    insert acc;
} catch (DmlException e) {
    System.debug('Error: ' + e.getMessage());
}
```

Explanation:

- Insert fails due to missing Name.
- Exception is caught, and error message is logged.



Database Methods vs. DML Statements

Database Methods provide:

- `allOrNone = false` to allow partial success.
- Detailed results (`Database.SaveResult`).

Example:

```
apex

Account acc1 = new Account();
Account acc2 = new Account(Name='Valid Account');

Database.SaveResult[] results = Database.insert(new List<Account>{acc1, acc2}, false);

for(Database.SaveResult sr : results){
    if(sr.isSuccess()){
        System.debug('Success! Id: ' + sr.getId());
    } else {
        for(Database.Error err : sr.getErrors()){
            System.debug('Error: ' + err.getMessage());
        }
    }
}
```

Explanation:

- Inserts 2 Accounts.
- `acc1` fails (no Name), `acc2` succeeds.
- No exception is thrown because `allOrNone=false`.



Key Considerations

- **Governor Limits:**
 - Max 150 DML statements per transaction
 - Max 10,000 records processed in total
- Always **bulkify** DML operations
- Use **try-catch** for error handling
- Use **Database** methods for finer control



Summary Table

Operation	Purpose	Example
insert	Add new records	insert acc;
update	Modify existing records	update acc;
upsert	Insert or update	upsert acc External_Id__c;
delete	Move records to Recycle Bin	delete acc;
undelete	Restore deleted records	undelete acc;
merge	Combine duplicates	merge master duplicate;


Would you like help with bulk DML examples, best practices, or writing Apex triggers using DML? Just let me know!